TRƯỜNG ĐẠI HỌC LẠC HỒNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KỲ
HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

ĐỀ TÀI: TÌM ĐƯỜNG ĐI NGẮN NHẤT TRÊN BẢN ĐỒ BẰNG THUẬT TOÁN DIJKSTRA VÀ A*

Cài đặt thuần cấu trúc Min-Heap có mảng chỉ mục, Bảng băm DJB2 và thực nghiệm đa quy mô

Giảng viên hướng dẫn: ThS. Đoàn Thiện Minh
Sinh viên thực hiện:
1. Nguyễn Hoàng Anh - MSSV: 125000977 (Nhóm trưởng)
2. Hà Văn Kiên - MSSV: 125000399
3. Trần Quang Huy - MSSV: 125001825
4. Hoàng Minh Phát - MSSV: 125001323

Lớp: 25CT111

Đồng Nai, tháng 11 năm 2026

LỜI CAM ĐOAN

Chúng em xin cam đoan toàn bộ nội dung công trình nghiên cứu và báo cáo đồ án này là kết quả lao động học thuật trung thực của tập thể nhóm dưới sự chỉ dẫn của giảng viên phụ trách học phần Cấu trúc Dữ liệu và Giải thuật. Mọi số liệu thực nghiệm, bảng số đo benchmark, biểu đồ và mã nguồn cài đặt trình bày trong tài liệu này đều được trích xuất từ quá trình chạy thực tế trên máy tính thử nghiệm, không qua bất kỳ hình thức can thiệp chỉnh sửa số liệu hay ngụy tạo kết quả.

Nhóm cam kết không sao chép nguyên văn, không chiếm dụng mã nguồn hay bài viết của bất kỳ cá nhân, đồ án khóa trước hay tài liệu trực tuyến nào mà không thực hiện trích dẫn khoa học minh bạch. Tất cả các cấu trúc dữ liệu nền tảng gồm Hàng đợi ưu tiên Min-Heap có mảng chỉ vị pos[], Bảng băm đóng dò tuyến tính giải quyết xung đột, Ngăn xếp truy vết và Hàng đợi kiểm tra liên thông đều được nhóm tự tay thiết kế và lập trình thuần túy mà không sử dụng bất kỳ thư viện hỗ trợ cấu trúc dữ liệu dựng sẵn nào.

Nếu có bất kỳ sự sai lệch hoặc vi phạm quy chế liêm chính học thuật nào bị phát hiện, toàn thể các thành viên trong nhóm xin chịu hoàn toàn trách nhiệm trước Hội đồng thẩm định và Quy định xử lý kỷ luật học vụ của Trường Đại học Lạc Hồng.

Đồng Nai, ngày 06 tháng 11 năm 2026
Đại diện nhóm sinh viên thực hiện
(Ký và ghi rõ họ tên)

Nguyễn Hoàng Anh

MỤC LỤC

No table of contents entries found.

DANH MỤC HÌNH VẼ

No table of figures entries found.

DANH MỤC BẢNG BIỂU

No table of figures entries found.

LỜI MỞ ĐẦU

Trong thời đại chuyển đổi số và phát triển đô thị thông minh, nhu cầu tối ưu hóa hành trình di chuyển đóng vai trò then chốt trong việc giảm thiểu tiêu hao nhiên liệu, bảo vệ môi trường và giải tỏa áp lực giao thông đô thị. Bài toán tìm đường đi ngắn nhất trên mạng lưới đường bộ thực tế là một bài toán nền tảng kinh điển của khoa học máy tính, đòi hỏi sự kết hợp chặt chẽ giữa các cấu trúc dữ liệu tiên tiến và các chiến lược giải thuật tối ưu hóa đồ thị.

Mục tiêu trọng tâm của đồ án là thiết kế, cài đặt thuần túy và kiểm chứng thực nghiệm hệ thống tìm đường đi ngắn nhất ứng dụng hai thuật toán nền tảng Dijkstra và A* trên đồ thị có hướng mang trọng số không âm. Toàn bộ hệ thống được xây dựng với nguyên tắc không phụ thuộc vào bất kỳ thư viện ngoài nào đối với các cấu trúc dữ liệu lõi, giúp người học nắm bắt tường tận từ bản chất cấu trúc bộ nhớ, cơ chế quản lý con trỏ, phép toán băm giải quyết xung đột đến kỹ thuật giảm khóa tối ưu trong hàng đợi ưu tiên.

Phạm vi nghiên cứu bao gồm việc mô hình hóa mạng lưới giao thông đô thị dạng đồ thị danh sách kề với tọa độ địa lý chuẩn hóa ki-lô-mét, hỗ trợ các tuyến đường một chiều, hai chiều và mô phỏng hệ số kẹt xe động theo thời gian thực. Bên cạnh đó, đồ án thực hiện đánh giá thực nghiệm toàn diện trên các tập dữ liệu từ bản đồ mẫu đô thị 15 nút đến các đồ thị lưới quy mô lớn đạt ngưỡng 20.000 đỉnh nhằm đối chiếu trực tiếp giữa lý thuyết độ phức tạp tiệm cận và thời gian thực thi đo đạc trên hệ thống phần cứng thực tế.

Bố cục báo cáo được tổ chức thành năm chương chính: Chương 1 trình bày bài toán và mô hình hóa hệ thống; Chương 2 phân tích sâu cơ sở lý thuyết cấu trúc dữ liệu và giải thuật; Chương 3 mô tả kiến trúc thiết kế và giải pháp cài đặt chi tiết kèm mã nguồn; Chương 4 minh họa từng bước chạy thuật toán trên bản đồ thực tế; và Chương 5 tổng kết toàn bộ kiểm thử đúng đắn cùng thực nghiệm benchmark hiệu năng chuyên sâu.

CHƯƠNG 1. BÀI TOÁN VÀ MÔ HÌNH HÓA

Bài toán đường đi ngắn nhất trên đồ thị có hướng là bài toán tìm kiếm một chuỗi liên tiếp các cung nối từ đỉnh xuất phát s đến đỉnh đích t sao cho tổng trọng số hiệu dụng tích lũy trên các cung là nhỏ nhất. Trong ngữ cảnh bản đồ giao thông đường bộ, một mạng lưới đường sá được mô hình hóa tự nhiên bằng đồ thị có hướng G = (V, E), trong đó tập đỉnh V biểu diễn tập hợp các giao lộ, bến xe, công viên hoặc các địa điểm trọng yếu, còn tập cung E đại diện cho các tuyến đường kết nối trực tiếp giữa các địa điểm đó. Mỗi cung có hướng e = (u, v) được gán một trọng số cơ sở w(u, v) biểu thị khoảng cách vật lý thực tế được đo lường bằng đơn vị ki-lô-mét.

Để phản ánh chính xác quy hoạch giao thông đô thị hiện đại, hệ thống phân biệt tường minh giữa tuyến đường một chiều và hai chiều. Tuyến đường một chiều từ đỉnh u đến đỉnh v chỉ cho phép phương tiện lưu thông theo chiều thuận e = (u, v), do đó chỉ cung này xuất hiện trong danh sách liên kết kề của đỉnh u mà không có chiều ngược lại. Ngược lại, tuyến đường hai chiều giữa hai đỉnh được mô hình hóa đối xứng bằng hai cung riêng biệt (u, v) và (v, u) có cùng độ dài vật lý cơ sở, cho phép các thuật toán điều hướng dễ dàng xử lý tương thích mà không làm mất tính tổng quát.

Yếu tố biến động giao thông thực tế được trừu tượng hóa thông qua hệ số kẹt xe k >= 1 được gán động cho từng tuyến đường. Khi xảy ra tình trạng ùn tắc tại một đoạn đường cụ thể, trọng số hiệu dụng dùng cho việc tính toán điều hướng được xác định theo nguyên lý tỷ lệ thuận bằng tích số giữa chiều dài cơ sở và hệ số kẹt xe tương ứng:

$$weff_{(u,v)} = w(u,v) \times k(u,v),\ k(u,v) \geq 1.0 \quad (1)$$

Hệ số kẹt xe k = 1.0 tương ứng với trạng thái đường thông thoáng lý tưởng khi vận tốc phương tiện đạt mức thiết kế tối đa, trong khi k > 1.0 mô phỏng tình trạng lưu lượng xe tăng cao làm giảm tốc độ hành trình, tương đương với việc gia tăng chi phí thời gian di chuyển trên đoạn đường đó. Giới hạn k >= 1.0 đảm bảo tính nhất quán vật lý vì không có hiện tượng giao thông tự nhiên nào làm thời gian di chuyển giảm xuống dưới ngưỡng vận tốc thiết kế tối đa của cung đường.

Trong mô hình toán học của đề tài, các cạnh mang trọng số âm hoàn toàn bị loại bỏ bởi hai lý do cơ bản. Về mặt vật lý giao thông, quãng đường di chuyển và thời gian tiêu tốn giữa hai địa điểm luôn là các đại lượng thực dương, không thể tồn tại một cung đường mà việc đi qua nó lại làm giảm quãng

đường tích lũy trước đó của phương tiện. Về mặt lý thuyết tính toán, sự xuất hiện của cạnh âm sẽ phá vỡ bất biến tham lam nền tảng của thuật toán Dijkstra, đồng thời nếu tồn tại chu trình có tổng trọng số âm thì đường đi ngắn nhất sẽ trở nên không xác định vì phương tiện có thể lặp vô hạn quanh chu trình để chi phí giảm về âm vô cực.

Để đảm bảo đồ án giải quyết trọn vẹn và chuẩn mực yêu cầu học thuật của đề tài, toàn bộ hệ thống được xây dựng nhằm đáp ứng toàn diện năm yêu cầu cốt lõi. Yêu cầu thứ nhất về việc cài đặt thuần cấu trúc dữ liệu không dùng thư viện ngoài được hiện thực hóa bằng việc tự lập trình IndexedMinHeap, CustomHashTable, CustomStack và CustomQueue. Yêu cầu thứ hai về việc cài đặt thuật toán Dijkstra và A* được hiện thực hóa trong hai mô-đun độc lập kèm cơ chế dừng sớm khi chạm đích. Yêu cầu thứ ba về mô phỏng mạng lưới giao thông thực tế được đáp ứng thông qua bản đồ 15 giao lộ có tọa độ hình học, đường một chiều và cập nhật kẹt xe động. Yêu cầu thứ tư về việc xây dựng giao diện trực quan được đáp ứng bằng cả giao diện dòng lệnh phân cấp và ứng dụng web mô phỏng tương tác. Yêu cầu thứ năm về kiểm nghiệm và phân tích hiệu năng được hiện thực hóa bằng bộ kiểm thử tự động 26 ca kiểm thử cùng hệ thống benchmark stress-test đa quy mô từ 100 đến 20.000 đỉnh.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT THUẬT TOÁN

2.1. Đồ thị và cách biểu diễn

Trong lý thuyết đồ thị và khoa học máy tính, hai phương pháp biểu diễn đồ thị phổ biến nhất là ma trận kề (adjacency matrix) và danh sách kề (adjacency list). Ma trận kề sử dụng một mảng hai chiều kích thước V × V, trong đó phần tử tại vị trí hàng u và cột v lưu trữ trọng số của cung nối từ u đến v, hoặc nhận giá trị vô cùng nếu không tồn tại cạnh nối trực tiếp. Phương pháp này có ưu điểm cho phép kiểm tra sự tồn tại của một cung bất kỳ trong thời gian hằng số O(1), song nhược điểm chí mạng là đòi hỏi không gian bộ nhớ cố định $O(V^2)$, bất kể số lượng cạnh thực tế nhiều hay ít.

Ngược lại, danh sách kề biểu diễn đồ thị bằng một mảng gồm V phần tử, trong đó mỗi phần tử chứa con trỏ trỏ đến đầu một danh sách liên kết đơn lưu trữ tập hợp các đỉnh kề trực tiếp cùng thuộc tính trọng số tương ứng. Tổng bộ nhớ tiêu thụ của danh sách kề chỉ tỷ lệ thuận với số lượng đỉnh và số lượng cung thực tế là O(V + E). Trong các mạng lưới giao thông đường bộ đô thị, đồ thị luôn mang bản chất thưa rõ rệt khi mỗi nút giao thông thông thường chỉ kết nối từ 2 đến 4 tuyến đường rẽ, dẫn đến số cạnh xấp xỉ tuyến tính với số đỉnh E xấp xỉ k·V với k <= 4. Với quy mô 20.000 đỉnh, ma trận kề đòi hỏi bảng 400 triệu ô nhớ gây lãng phí nghiêm trọng, trong khi danh sách kề chỉ yêu cầu khoảng 100.000 phần tử liên kết, do đó danh sách kề là sự lựa chọn tối ưu tuyệt đối cho đồ án.

2.2. Hàng đợi ưu tiên và Min-Heap

Hàng đợi ưu tiên (priority queue) là cấu trúc dữ liệu trừu tượng quản lý tập hợp các phần tử có gắn kèm khóa độ ưu tiên, hỗ trợ trích xuất phần tử có khóa nhỏ nhất một cách hiệu quả. Min-Heap là một hiện thực kinh điển của hàng đợi ưu tiên dưới dạng cây nhị phân gần đầy đủ (nearly complete binary tree), thỏa mãn tính chất đống: khóa của mỗi nút cha luôn nhỏ hơn hoặc bằng khóa của hai nút con tương ứng. Cấu trúc này được lưu trữ liên tục trên một mảng một chiều tuyến tính mà không cần cấp phát con trỏ động, giúp tiết kiệm bộ nhớ và tối ưu hóa bộ nhớ đệm CPU.

Với quy ước chỉ số mảng bắt đầu từ 0, mối quan hệ phân cấp giữa chỉ số nút cha và các nút con được xác định hoàn toàn thông qua các công thức số học nguyên:

$$parent(i) = \left\lfloor \frac{i - 1}{2} \right\rfloor \quad (2)$$

$$left_child(i) = 2i + 1 \quad (3)$$

$$right_child(i) = 2i + 2 \quad (4)$$

Thao tác chèn một phần tử mới insert(priority, item) được thực hiện bằng cách bổ sung phần tử vào cuối mảng rồi kích hoạt thủ tục vun lên sift-up. Thao tác trích xuất đỉnh nhỏ nhất pop_min() lấy phần tử gốc tại chỉ số 0, đưa phần tử cuối cùng lên thay thế gốc và kích hoạt thủ tục vun xuống sift-down để tái lập bất biến đống. Vì cây nhị phân gần đầy đủ chứa n phần tử có chiều cao h được chặn trên bởi h = floor(log2(n)), số lượng phép tráo đổi nút trong quá trình vun lên hoặc vun xuống dọc theo một đường

đi từ gốc tới lá không bao giờ vượt quá chiều cao cây, chứng minh độ phức tạp thời gian của cả hai thao tác đều đạt O(log n).

Trong thuật toán tìm đường đi ngắn nhất, khi một tuyến đường tốt hơn tới đỉnh v được phát hiện, ta cần cập nhật giảm khoảng cách của đỉnh v trong hàng đợi ưu tiên, thao tác này được gọi là decrease-key. Nếu sử dụng heap tiêu chuẩn, việc định vị đỉnh v trong heap đòi hỏi quét tuyến tính toàn bộ mảng mất thời gian O(n), làm suy thoái độ phức tạp của thuật toán Dijkstra lên $O(V^2)$. Để giải quyết triệt để vấn đề này, đề tài cài đặt IndexedMinHeap tích hợp một mảng vị trí pos[] kích thước V, trong đó pos[v] lưu trữ chính xác chỉ số vị trí hiện tại của đỉnh v trong mảng heap, cho phép tra cứu tọa độ đỉnh trong O(1) và thực hiện vun lên sift-up trong O(log V). Cách tiếp cận này vượt trội hoàn toàn so với kỹ thuật chèn trùng nút (lazy deletion) vốn làm kích thước heap phình to lên O(E) phần tử và tiêu tốn thêm thời gian xử lý các bản ghi rác.

2.3. Thuật toán Dijkstra

Thuật toán Dijkstra do nhà khoa học máy tính Edsger W. Dijkstra đề xuất năm 1959 là thuật toán tham lam kinh điển giải quyết bài toán đường đi ngắn nhất từ một nguồn đơn trên đồ thị có trọng số không âm. Thuật toán duy trì một mảng khoảng cách ước lượng dist[v] cho mọi đỉnh v, khởi tạo dist[s] = 0 cho đỉnh xuất phát s và dist[v] = vô cực cho tất cả các đỉnh còn lại. Tại mỗi bước lặp, thuật toán trích xuất đỉnh u có giá trị dist[u] nhỏ nhất trong số các đỉnh chưa hoàn tất, chốt đỉnh u là tối ưu, sau đó thực hiện thao tác nới lỏng (relaxation) trên tất cả các cung đi ra (u, v) theo công thức toán học:

$$dist[v] \leftarrow min(dist[v], dist[u] + weff_{(u,v)}) \quad (5)$$

Tính đúng đắn của thuật toán Dijkstra được chứng minh chặt chẽ bằng phương pháp quy nạp toán học dựa trên bất biến vòng lặp: tại thời điểm một đỉnh u bất kỳ được trích xuất ra khỏi hàng đợi ưu tiên, giá trị dist[u] đã đạt tới khoảng cách ngắn nhất tuyệt đối delta(s, u). Cơ sở quy nạp đúng tại bước khởi tạo khi đỉnh s được lấy ra đầu tiên với dist[s] = 0. Giả sử bất biến đúng cho tập hợp các đỉnh đã chốt S. Khi chọn đỉnh kế tiếp u không thuộc S có dist[u] nhỏ nhất, giả sử phản chứng tồn tại một đường đi thực tế P nối s tới u có tổng chiều dài ngắn hơn dist[u]. Khi đó đường đi P phải rời tập S tại một cung nào đó (x, y) với x thuộc S và y chưa thuộc S. Vì x đã được chốt tối ưu và đã nới lỏng cạnh (x, y), ta có dist[y] <= chiều dài đoạn đường từ s đến y trên P. Do trọng số mọi cung đều không âm, chiều dài từ s đến y không vượt quá chiều dài toàn bộ đường đi P, dẫn tới dist[y] <= length(P) < dist[u]. Tuy nhiên điều này mâu thuẫn trực tiếp với quy tắc chọn đỉnh tham lam rằng u là đỉnh chưa chốt có dist nhỏ nhất trong hàng đợi ưu tiên, chứng tỏ không thể tồn tại đường đi P nào ngắn hơn dist[u].

Về mặt độ phức tạp tính toán, thuật toán thực hiện chính xác V lần thao tác trích xuất đỉnh pop_min() và nhiều nhất E lần thao tác nới lỏng cạnh dẫn đến thao tác giảm khóa decrease_key() trên IndexedMinHeap. Do mỗi thao tác trên heap nhị phân có chi phí thời gian O(log V), tổng thời gian thực thi của thuật toán đạt mức chặn tiệm cận O((V + E) log V).

Giả thiết trọng số không âm đóng vai trò sinh tử đối với tính đúng đắn của phép chứng minh quy nạp nói trên. Khi xuất hiện cạnh mang trọng số âm, một đường đi vòng qua đỉnh khác có thể tạo ra tổng chi phí nhỏ hơn khoảng cách của đỉnh đã chốt, khiến bất biến bị phá vỡ hoàn toàn. Ví dụ, xét đồ thị ba đỉnh gồm cung (A, B) có trọng số 3, cung (A, C) có trọng số 5 và cung (C, B) có trọng số âm bằng -4. Thuật toán Dijkstra sẽ tham lam chốt đỉnh B ngay từ bước đầu với khoảng cách dist[B] = 3 vì 3 < 5. Tuy nhiên, đường đi thực tế A -> C -> B lại có tổng chi phí bằng 5 + (-4) = 1, nhỏ hơn 3, chứng minh tính đúng đắn bị phá hỏng hoàn toàn khi tồn tại trọng số âm.

2.4. Thuật toán A* và tính Admissible của Heuristic

Thuật toán A* do Peter Hart, Nils Nilsson và Bertram Raphael công bố năm 1968 là thuật toán tìm kiếm theo thông tin (informed search) mở rộng từ Dijkstra nhằm định hướng không gian tìm kiếm về phía đỉnh mục tiêu t. Thuật toán đánh giá độ ưu tiên của mỗi đỉnh n thông qua hàm mục tiêu tổng hợp f(n) kết hợp giữa chi phí đã tích lũy và chi phí ước lượng tương lai:

$$f(n) = g(n) + h(n) \quad (6)$$

Trong đó g(n) biểu diễn chi phí thực tế nhỏ nhất từ đỉnh xuất phát s đến đỉnh n đã tìm được, còn h(n) là hàm heuristic đánh giá khoảng cách ước lượng từ đỉnh n đến đỉnh đích t. Trên bản đồ tọa độ phẳng, hàm heuristic tự nhiên nhất là khoảng cách đường chim bay Euclid giữa hai điểm tọa độ.

Khoảng cách Euclid luôn là cận dưới toán học của quãng đường thực tế vì đường thẳng luôn là khoảng cách ngắn nhất giữa hai điểm trong không gian phẳng.

Một hàm heuristic h(n) được gọi là chấp nhận được (admissible) nếu với mọi đỉnh n, giá trị ước lượng h(n) không bao giờ vượt quá khoảng cách ngắn nhất thực tế h*(n) từ n đến đỉnh đích t, tức thỏa mãn:

$$0 \leq h(n) \leq h*(n), \ \forall n \in V \tag{7}$$

Hàm heuristic h(n) được gọi là nhất quán (consistent hay monotone) nếu với mọi cung (u, v) trên đồ thị, giá trị ước lượng thỏa mãn bất đẳng thức tam giác:

$$h(u) \leq weff_{(u,v)} + h(v), \ h(t) = 0 \tag{8}$$

Khi hàm heuristic thỏa mãn tính nhất quán, giá trị hàm f(n) luôn đơn điệu không giảm dọc theo mọi đường đi hợp lệ. Khi một đỉnh u được trích xuất ra khỏi hàng đợi ưu tiên, giá trị g(u) được đảm bảo đã đạt mức chi phí tối ưu tuyệt đối mà không bao giờ cần phải mở lại (re-open) đỉnh đó, giúp thuật toán A* bảo toàn trọn vẹn kết quả đường đi ngắn nhất hoàn toàn trùng khớp với thuật toán Dijkstra. Bản thân thuật toán Dijkstra thực chất chính là một trường hợp đặc biệt của thuật toán A* khi hàm heuristic h(n) = 0 với mọi đỉnh n.

Trong các hệ thống bản đồ cho phép người dùng tự do chỉnh sửa hoặc nhập trọng số cạnh thủ công, có thể xảy ra trường hợp trọng số cơ sở của một cung nhỏ hơn khoảng cách hình học Euclid giữa hai đầu mút của nó. Nếu giữ nguyên khoảng cách Euclid thuần túy làm hàm đánh giá, hàm heuristic sẽ vượt trần khoảng cách thực tế và đánh mất tính admissible, dẫn đến nguy cơ A* đưa ra lộ trình sai lệch. Để triệt tiêu hoàn toàn rủi ro này, chương trình tự động tính toán một hệ số tỷ lệ co giãn an toàn h_scale theo công thức cực tiểu hóa trên toàn bộ tập cung của đồ thị:

$$h_scale = max\left(0, \quad min\left(1.0, \quad \min_{(u,v)\in E, d_E > 0} \frac{w(u,v)}{dEuclid_{(u,v)}}\right)\right) \tag{9}$$

Nhờ có hệ số h_scale, hàm heuristic h(n) = h_scale × d_Euclid(n, t) luôn được bảo đảm là một cận dưới chặt chẽ của quãng đường thực tế trong mọi tình huống. Tuy nhiên, thuật toán A* không phải lúc nào cũng mang lại lợi thế vượt trội. Trong các đồ thị dạng mê cung có nhiều vật cản hoặc đồ thị có hàm heuristic quá yếu (h_scale xấp xỉ 0), vector hướng đi hình học bị cản trở bởi các ngõ cụt buộc thuật toán phải quay lui liên tục, khi đó số lượng đỉnh duyệt của A* sẽ tương đương với Dijkstra nhưng tổng thời gian chạy thực tế có thể lớn hơn do phải tốn thêm chi phí tính toán căn bậc hai cho mỗi lân cận.

2.5. Các cấu trúc phụ trợ và thuật toán số học

Nhằm đảm bảo tôn chỉ không phụ thuộc vào thư viện ngoài, hệ thống đã tự xây dựng ba cấu trúc và giải thuật số học phụ trợ thiết yếu. Cấu trúc thứ nhất là bảng băm CustomHashTable sử dụng hàm băm DJB2 do Daniel J. Bernstein đề xuất kết hợp kỹ thuật giải quyết xung đột bằng phương pháp dò tuyến tính (linear probing). Hàm băm DJB2 khởi tạo giá trị băm h = 5381 và liên tục nhân với 33 kết hợp cộng mã ký tự tiếp theo:

$$hashi = (hashi - 1 \times 33 + ci) \ mod \ Capacity \tag{10}$$

Để kiểm soát xung đột khi bảng băm đầy dần, bảng băm tự động giám sát hệ số tải alpha = size / capacity. Khi alpha >= 0.5, hàm _rehash() được kích hoạt để mở rộng kích thước mảng lên capacity mới = 2 × capacity cũ + 1 và băm lại toàn bộ các khóa hiện có. Nhờ kiểm soát hệ số tải dưới ngưỡng 0.5, độ phức tạp trung bình cho các thao tác tra cứu, thêm và kiểm tra tồn tại luôn được duy trì ở mức O(1), trong khi trường hợp xấu nhất khi toàn bộ các khóa rơi vào cùng một chuỗi cụm là O(n).

Cấu trúc thứ hai là ngăn xếp CustomStack hoạt động theo cơ chế Vào sau - Ra trước (LIFO). Ngăn xếp đóng vai trò tối quan trọng trong việc phục hồi đường đi tối ưu: mảng prev[] của thuật toán Dijkstra và A* lưu vết chỉ số đỉnh cha đi trước, do đó quá trình truy vết ngược từ đỉnh đích t về đỉnh xuất phát s sẽ thu được thứ tự các đỉnh bị đảo ngược. Bằng cách đẩy lần lượt các đỉnh vào CustomStack và sau đó rút tuần tự ra danh sách, lộ trình hoàn chỉnh từ điểm xuất phát đến điểm kết thúc được phục hồi chính xác trong độ phức tạp thời gian tuyến tính O(V).

Thuật toán số học thứ ba là phép tính căn bậc hai số thực sqrt_newton phục vụ tính khoảng cách Euclid mà không sử dụng hàm thư viện math.sqrt. Giải thuật áp dụng phương pháp lặp tiếp tuyến

Newton-Raphson để giải phương trình f(x) = x² - S = 0 với S là bình phương khoảng cách, thông qua công thức lặp hồi quy:

$$xn+1 \quad = \frac{1}{2}\left(xn \quad + \frac{S}{xn}\right) \tag{11}$$

Vòng lặp kết thúc khi độ chênh lệch tuyệt đối giữa hai bước lặp liên tiếp đạt ngưỡng dung sai |x_n+1 - x_n| < 10^(-7). Nhờ đặc tính hội tụ bậc hai (quadratic convergence) của phương pháp Newton-Raphson khi số lượng chữ số có nghĩa chính xác tăng gấp đôi sau mỗi bước lặp, thuật toán đạt độ chính xác tương đương kiểu số thực kép chỉ sau khoảng 5 đến 7 phép lặp, đảm bảo tính toán siêu nhanh và tin cậy tuyệt đối.

CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

Hệ thống tìm đường đi ngắn nhất được thiết kế theo kiến trúc hướng đối tượng mô-đun hóa cao, phân định rõ ràng giữa các cấu trúc dữ liệu nền tảng, các giải thuật điều hướng cốt lõi và hệ thống giao tiếp người dùng. Toàn bộ các lớp và hàm được đặt tên tường minh, đúng chuẩn quy ước kỹ thuật và được cài đặt thuần túy tại thư mục mã nguồn NopBai/src/.

3.1. Các lớp cấu trúc dữ liệu nền tảng

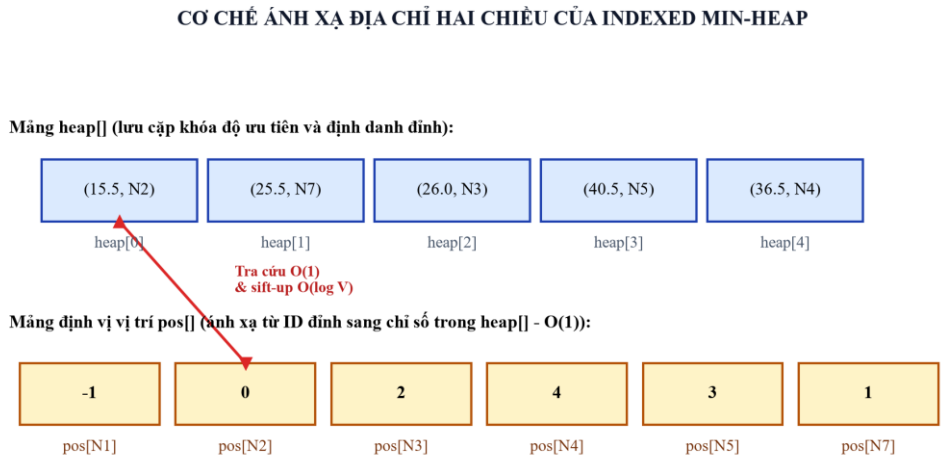
Lớp CustomHashTable (trong file src/cautruc/hashtable.py) quản lý ánh xạ hai chiều giữa chuỗi định danh địa điểm (tên giao lộ) và chỉ số số nguyên đại diện trong đồ thị. Bảng băm lưu trữ hai mảng song song self.keys và self.values cùng biến đếm self.size. Khi thêm một cặp khóa - giá trị bằng phương thức put(khoa, gia_tri), thuật toán tính chỉ số băm qua DJB2 và thực hiện dò tuyến tính. Nếu hệ số tải vượt ngưỡng 50%, phương thức _rehash() tự động nhân đôi kích thước bảng và băm lại toàn bộ dữ liệu, duy trì độ phức tạp trung bình O(1) cho phương thức get(khoa) và contains(khoa).

Lớp IndexedMinHeap (trong file src/cautruc/minheap.py) cài đặt hàng đợi ưu tiên có mảng định vị chỉ số. Dữ liệu đống được lưu trong self.heap dưới dạng danh sách các cặp (priority, item), kết hợp mảng self.pos có kích thước cố định bằng capacity đỉnh. Thao tác insert(priority, item) thêm phần tử vào cuối đống, ghi nhận vị trí vào self.pos và kích hoạt _sift_up(). Thao tác decrease_key(item, new_priority) sử dụng mảng self.pos để định vị ngay lập tức vị trí phần tử trong O(1), cập nhật độ ưu tiên mới và thực hiện vun lên đống trong O(log V). Thao tác pop_min() lấy phần tử gốc nhỏ nhất, hoán vị phần tử cuối lên thay thế và thực hiện _sift_down() trong O(log V). Mọi thao tác đều được theo dõi bằng bộ đếm self.heap_ops phục vụ đánh giá thực nghiệm.

```
def decrease_key(self, item: int, new_priority: float) -> None:
    self.heap_ops += 1
    idx = self.pos[item]
    if idx != -1 and new_priority < self.heap[idx][0]:
        self.heap[idx] = (new_priority, item)
        self._sift_up(idx)

def pop_min(self) -> Tuple[Optional[float], Optional[int]]:
    self.heap_ops += 1
    if not self.heap: return None, None
    min_prio, min_item = self.heap[0]
    self.pos[min_item] = -1
    last_prio, last_item = self.heap.pop()
    if self.heap:
        self.heap[0] = (last_prio, last_item)
        self.pos[last_item] = 0
        self._sift_down(0)
    return min_prio, min_item
```

Đoạn mã trên minh họa sự phối hợp nhịp nhàng giữa mảng pos[] và mảng heap[]. Trong decrease_key, việc tra cứu trực tiếp vị trí idx = self.pos[item] giúp triệt tiêu hoàn toàn thao tác duyệt mảng tốn kém O(n). Trong pop_min, việc xóa gốc và cập nhật lại chỉ số pos[last_item] = 0 trước khi thực hiện sift-down đảm bảo tính toàn vẹn địa chỉ của mọi nút trong đống.



Hình 3.1. Sơ đồ cơ chế quản lý con trỏ và mảng định vị vị trí hai chiều trong IndexedMinHeap

Lớp CustomStack (trong file src/cautruc/stack.py) và CustomQueue (trong file src/cautruc/queue_.py) cung cấp các thao tác cơ bản phục vụ điều hướng. CustomStack cài đặt push, pop và to_list() trong O(1) mỗi thao tác, dùng để đảo chiều mảng dấu vết prev[] nhằm phục hồi đường đi từ điểm xuất phát tới điểm đích. CustomQueue sử dụng kỹ thuật mảng hai con trỏ quản lý chỉ số head trong O(1) thời gian trích xuất, phục vụ thuật toán duyệt theo chiều rộng BFS kiểm tra tính liên thông của mạng lưới đường bộ.

3.2. Cấu trúc đồ thị CityMapGraph

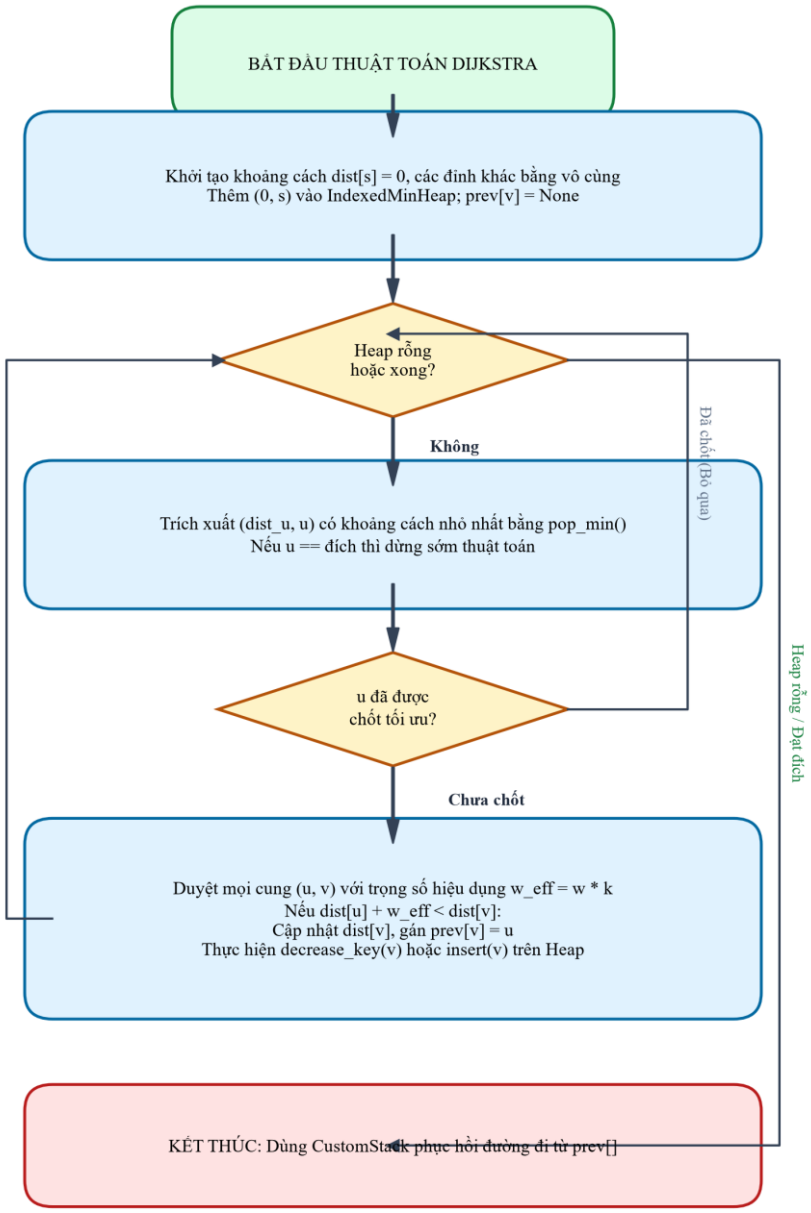
Lớp CityMapGraph (trong file src/cautruc/graph.py) là trung tâm biểu diễn mạng lưới giao thông đô thị. Mỗi tuyến đường đi ra từ một nút được đóng gói trong một đối tượng AdjListNode gồm chỉ số đích target_idx, trọng số cơ sở base_weight, hệ số kẹt xe traffic_factor, cờ báo một chiều is_oneway và con trỏ next liên kết tới cung tiếp theo. Phương thức get_effective_weight() trả về tích số giữa trọng số cơ sở và hệ số kẹt xe.

Lớp CityMapGraph duy trì danh sách tên đỉnh self.node_names, danh sách tọa độ phẳng self.node_coords lưu cặp (x, y) bằng ki-lô-mét, bảng băm tra cứu self.node_lookup và mảng danh sách kề self.adj. Phương thức add_node(name, x, y) đăng ký đỉnh mới hoặc cập nhật tọa độ hình học nếu đỉnh đã tồn tại. Phương thức add_edge(u_name, v_name, base_weight, traffic_factor, is_oneway) kiểm tra tính hợp lệ của trọng số cơ sở (từ chối trọng số âm), thêm cung thuận e = (u, v) và tự động thêm cung nghịch e = (v, u) nếu cung là đường hai chiều.

Phương thức update_traffic(u_name, v_name, traffic_factor) tiếp nhận hệ số kẹt xe k, kiểm tra điều kiện k >= 1.0 và cập nhật hệ số trên cung đường tương ứng. Phương thức compute_admissible_h_scale() quét qua toàn bộ các cung trên đồ thị để tính tỷ số nhỏ nhất giữa trọng số cơ sở và khoảng cách Euclid, đảm bảo tính admissible tuyệt đối cho hàm heuristic của thuật toán A*.

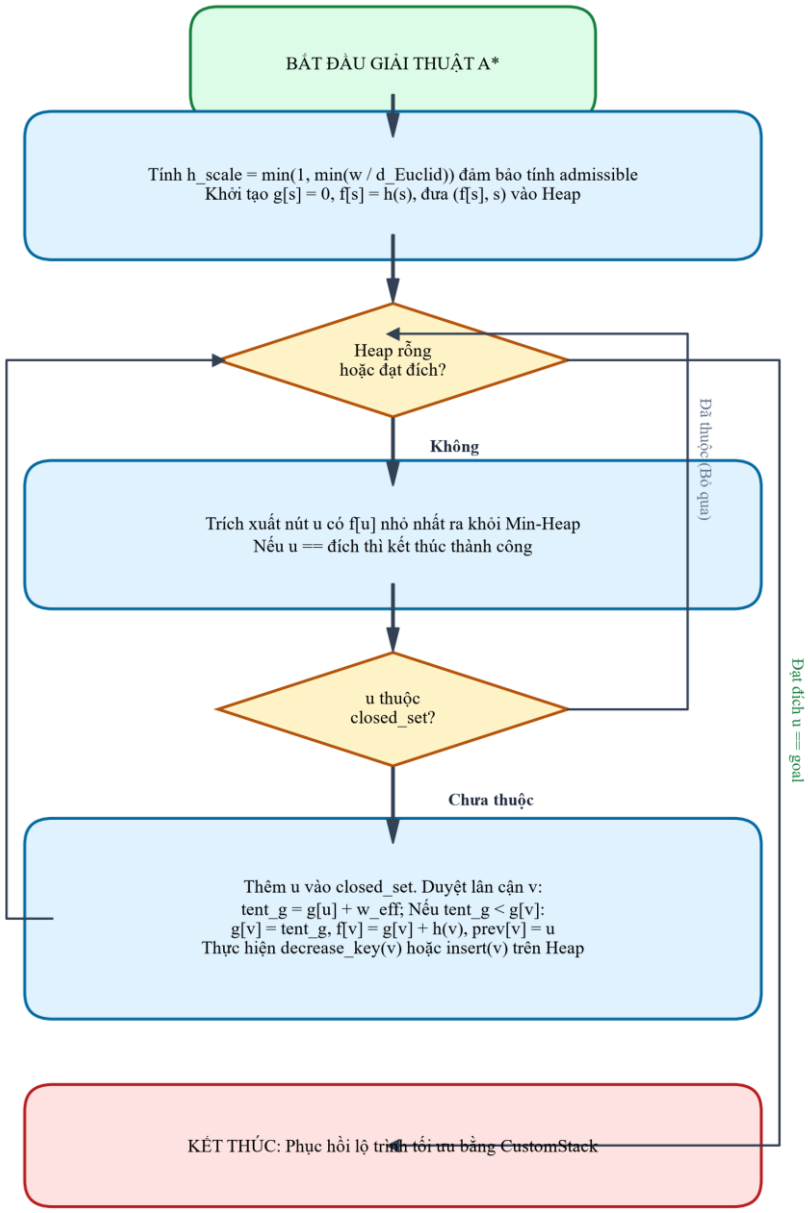
3.3. Cài đặt thuật toán Dijkstra và A*

Hàm dijkstra_shortest_path (trong file src/thuattoan/dijkstra.py) tiếp nhận đối tượng CityMapGraph cùng tên đỉnh xuất phát và đỉnh đích tùy chọn. Hàm khởi tạo mảng dist[] bằng vô cực, dist[start_idx] = 0 và đưa (0, start_idx) vào IndexedMinHeap. Vòng lặp chính liên tục rút đỉnh u có khoảng cách nhỏ nhất, kiểm tra cờ visited[u] để tránh duyệt lặp, và dừng sớm ngay khi u trùng với end_idx. Quá trình nới lỏng duyệt danh sách liên kết của đỉnh u, tính new_dist = dist[u] + w_eff, cập nhật mảng dist[v], mảng prev[v] và gọi decrease_key hoặc insert trên heap. Kết quả trả về được đóng gói chuẩn hóa trong đối tượng KetQuaTimDuong.



Hình 3.2. Lưu đồ giải thuật tìm đường đi ngắn nhất Dijkstra có cơ chế dừng sớm khi gặp đích

Hàm astar_shortest_path (trong file src/thuattoan/astar.py) thực hiện tìm đường có định hướng bằng hàm heuristic Euclid. Hàm tự động gọi dothi.compute_admissible_h_scale() để xác định hệ số co giãn an toàn h_scale trước khi khởi tạo hàm đánh giá heuristic. Hai mảng g_score[] và f_score[] được quản lý song song, trong đó f_score[v] = g_score[v] + h(v) được dùng làm khóa sắp xếp trong IndexedMinHeap. Khi duyệt cung (u, v), nếu chi phí tạm thời tent_g = g_score[u] + w_eff nhỏ hơn g_score[v], thuật toán cập nhật g_score[v], tính lại f_score[v] và kích hoạt decrease_key trên heap.



Hình 3.3. Lưu đồ giải thuật tìm kiếm tối ưu A kết hợp hàm Heuristic Euclid Admissible*

3.4. Xử lý các trường hợp biên và điều kiện bất thường

Hệ thống được thiết kế phòng thủ chặt chẽ nhằm xử lý an toàn mọi tình huống biên có thể phát sinh trong quá trình vận hành thực tế. Đối với trường hợp điểm bắt đầu trùng điểm kết thúc (start == end), cả Dijkstra và A* đều phát hiện tức thời tại khâu kiểm tra đầu vào, trả về lộ trình gồm duy nhất một đỉnh với tổng chi phí bằng 0.0, số đỉnh duyệt bằng 1 và số thao tác heap bằng 1 mà không cần kích hoạt vòng lặp chính. Đối với trường hợp truy vấn đỉnh không tồn tại, hàm tra cứu bảng băm trả về None và hệ thống phát sinh thông báo lỗi rõ ràng, trả về KetQuaTimDuong với cờ success = False mà không gây sập chương trình.

Đối với trường hợp đồ thị mất liên thông khi không tồn tại bất kỳ đường đi nào kết nối từ điểm xuất phát đến điểm đích, giá trị khoảng cách tại đích dist[end_idx] vẫn giữ nguyên giá trị vô cực sau khi hàng đợi ưu tiên đã vét cạn toàn bộ thành phần liên thông chứa đỉnh nguồn. Khi đó hệ thống phát hiện dist == inf và trả về thông báo an toàn báo hiệu hai địa điểm không thể kết nối. Đối với các dữ liệu đầu vào không hợp lệ như thêm cạnh có trọng số âm hoặc cập nhật hệ số kẹt xe k < 1.0, các hàm kiểm soát add_edge và update_traffic lập tức từ chối và gửi thông điệp cảnh báo, bảo toàn trọn vẹn mô hình vật lý và tính đúng đắn toán học của hệ thống.

CHƯƠNG 4. MINH HỌA THUẬT TOÁN TRÊN BẢN ĐỒ MẪU

Để kiểm chứng trực quan cơ chế hoạt động của thuật toán Dijkstra và A*, chương trình thực hiện chạy thực tế trên bản đồ đồ thị mẫu gồm 15 nút giao thông và 24 tuyến đường kết nối tại tỉnh Bến Tre (được khởi tạo bởi hàm khoi_tao_ban_do_mau_15_nut trong file src/tienich/dulieu.py). Trong mạng lưới này, tồn tại ba tuyến đường một chiều trọng yếu: tuyến từ Chợ Bến Tre (N3) đến Đại học Lạc Hồng (N12) dài 18.0 km, tuyến từ Bến xe Bến Tre (N6) đến Bệnh viện Nguyễn Đình Chiểu (N9) dài 16.0 km, và tuyến từ Đại học Lạc Hồng (N12) đến UBND Thành phố (N8) dài 16.0 km. Chúng ta chọn cặp điểm khảo sát từ Vòng xoay Bến Tre (N1) đến Đại học Lạc Hồng CS1 (N12), là cặp điểm chịu sự chi phối quyết định của tuyến đường một chiều N3 -> N12.

4.1. Bảng vết chạy từng bước của thuật toán Dijkstra

Thuật toán Dijkstra khởi động từ đỉnh xuất phát N1 với khoảng cách dist[N1] = 0.0. Dưới đây là bảng vết chi tiết ghi nhận chính xác trạng thái từng bước lặp, đỉnh được trích xuất khỏi heap, nội dung các đỉnh trong heap sau khi trích xuất, và các cập nhật khoảng cách diễn ra:

Bảng 4.1. Bảng vết thực thi từng bước của thuật toán Dijkstra trên cặp điểm N1 -> N12

Bước	Đỉnh trích xuất	Khoảng cách	Nội dung Min-Heap sau khi trích xuất	Cập nhật nới lỏng (dist, prev)
1	N1_VongXoay	0.0	[]	N2 (15.5, N1); N3 (26.0, N1); N7 (25.5, N1)
2	N2_DaiLoDK	15.5	[(25.5, N7), (26.0, N3)]	N4 (36.5, N2); N5 (40.5, N2)
3	N7_CauRM	25.5	[(26.0, N3), (40.5, N5), (36.5, N4)]	Không nới lỏng thêm đỉnh mới
4	N3_ChoBT	26.0	[(36.5, N4), (40.5, N5)]	N12 (44.0, N3); N13 (46.5, N3)
5	N4_CauBT1	36.5	[(40.5, N5), (46.5, N13), (44.0, N12)]	N6 (62.0, N4); N8 (52.5, N4)
6	N5_CongVienMH	40.5	[(44.0, N12), (46.5, N13), (52.5, N8), (62.0, N6)]	N6 giảm xuống 58.5 từ N5
7	N12_DaiHocLH	44.0	[(46.5, N13), (58.5, N6), (52.5, N8)]	Đạt đỉnh đích N12 -> Dừng thuật toán

Phân tích diễn tiến cho thấy tại bước 1, thuật toán trích xuất N1 và nới lỏng ba đỉnh kề N2 (15.5 km), N7 (25.5 km) và N3 (26.0 km). Tại bước 2, N2 có khoảng cách nhỏ nhất (15.5 km) được lấy ra, mở rộng tiếp đến N4 (36.5 km) và N5 (40.5 km). Tại bước 3, đỉnh N7 (25.5 km) được lấy ra nhưng không cải thiện được đường đi tới các nút lân cận. Đến bước 4, đỉnh N3 (26.0 km) được trích xuất và mở rộng dọc theo tuyến một chiều N3 -> N12 với chi phí 26.0 + 18.0 = 44.0 km, đồng thời mở tới N13 (46.5 km). Sau khi lần lượt xét duyệt N4 và N5 do khoảng cách của chúng (36.5 km và 40.5 km) vẫn nhỏ hơn 44.0 km, thuật toán trích xuất thành công đỉnh đích N12 tại bước 7 với chi phí tối ưu 44.0 km và dừng lại. Sử dụng CustomStack lần ngược mảng prev: prev[N12] = N3, prev[N3] = N1, ta thu được đường đi tối ưu ban đầu là N1 -> N3 -> N12.

4.2. Bảng vết chạy từng bước của thuật toán A*

Với cùng cặp điểm xuất phát N1 (tọa độ x=8.5, y=10.2) và đích N12 (tọa độ x=25.5, y=42.5), hàm compute_admissible_h_scale trả về h_scale = 1.0. Khoảng cách Euclid từ N1 đến N12 là 36.5 km. Thuật toán A* khởi tạo với g[N1] = 0.0, h[N1] = 36.5 km và đưa f[N1] = 36.5 vào Min-Heap. Bảng 4.2 ghi nhận toàn bộ quá trình chạy thực tế của thuật toán A*:

Bảng 4.2. Bảng vết thực thi từng bước của thuật toán A trên cặp điểm N1 -> N12*

Bước	Đỉnh trích xuất	g(u)	h(u)	f(u)	Nội dung Min-Heap sau pop	Cập nhật nới lỏng (g, h, f, prev)

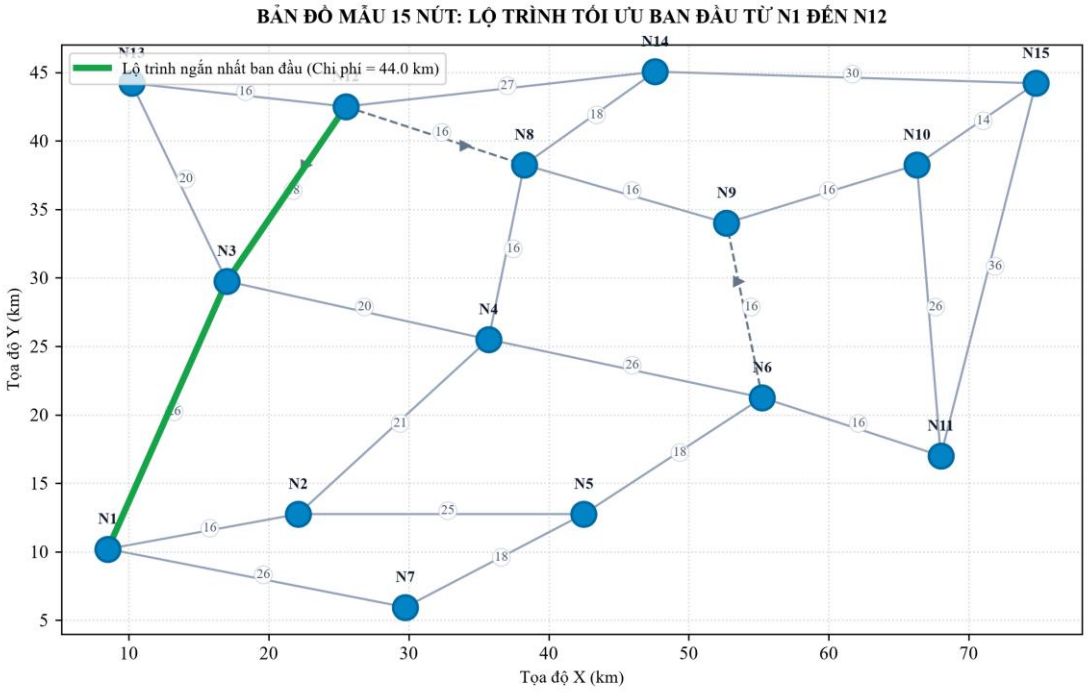
Bước	Đỉnh trích xuất	g(u)	h(u)	f(u)	Nội dung Min-Heap sau pop	Cập nhật nới lỏng (g, h, f, prev)
1	N1_VongXoay	0.0	36.5	36.5	[]	N2 (g=15.5, h=29.9, f=45.4); N3 (g=26.0, h=15.3, f=41.3); N7 (g=25.5, h=36.8, f=62.3)
2	N3_ChoBT	26.0	15.3	41.3	[(45.4, N2), (62.3, N7)]	N4 (g=46.5, h=19.8, f=66.3); N12 (g=44.0, h=0.0, f=44.0); N13 (g=46.5, h=15.4, f=61.9)
3	N12_DaiHocLH	44.0	0.0	44.0	[(45.4, N2), (61.9, N13), (66.3, N4), (62.3, N7)]	Đạt đỉnh đích N12 -> Dừng thuật toán thành công

Sự vượt trội của A* thể hiện rõ rệt ở cơ chế định hướng thông minh nhờ hàm heuristic. Tại bước 1, mặc dù đỉnh N2 có g(N2) = 15.5 km nhỏ hơn g(N3) = 26.0 km, nhưng do N3 nằm gần đích N12 hơn rất nhiều (h(N3) = 15.3 km so với h(N2) = 29.9 km), tổng hàm mục tiêu f(N3) = 41.3 km nhỏ hơn f(N2) = 45.4 km. Do đó, tại bước 2, Min-Heap trích xuất ngay đỉnh N3 mà bỏ qua N2. Tại N3, cung một chiều N3 -> N12 lập tức đưa đỉnh đích N12 vào heap với f(N12) = 44.0 + 0.0 = 44.0 km. Tại bước 3, f(N12) = 44.0 km nhỏ hơn f(N2) = 45.4 km nên N12 được trích xuất ngay lập tức, hoàn thành việc tìm kiếm mà hoàn toàn không cần duyệt qua các đỉnh N2, N4, N5 hay N7.

Đối chiếu trực tiếp trên ví dụ cụ thể này: Dijkstra phải duyệt qua 7 đỉnh và thực hiện 14 thao tác heap, trong khi A* chỉ duyệt duy nhất 3 đỉnh và thực hiện 8 thao tác heap, tiết kiệm tới 57.1% số đỉnh mở rộng mà vẫn đảm bảo tính ra chính xác tuyệt đối cùng một chi phí tối ưu 44.0 km và cùng một lộ trình tối ưu.

4.3. Mô phỏng kẹt xe động và tự động chuyển hướng

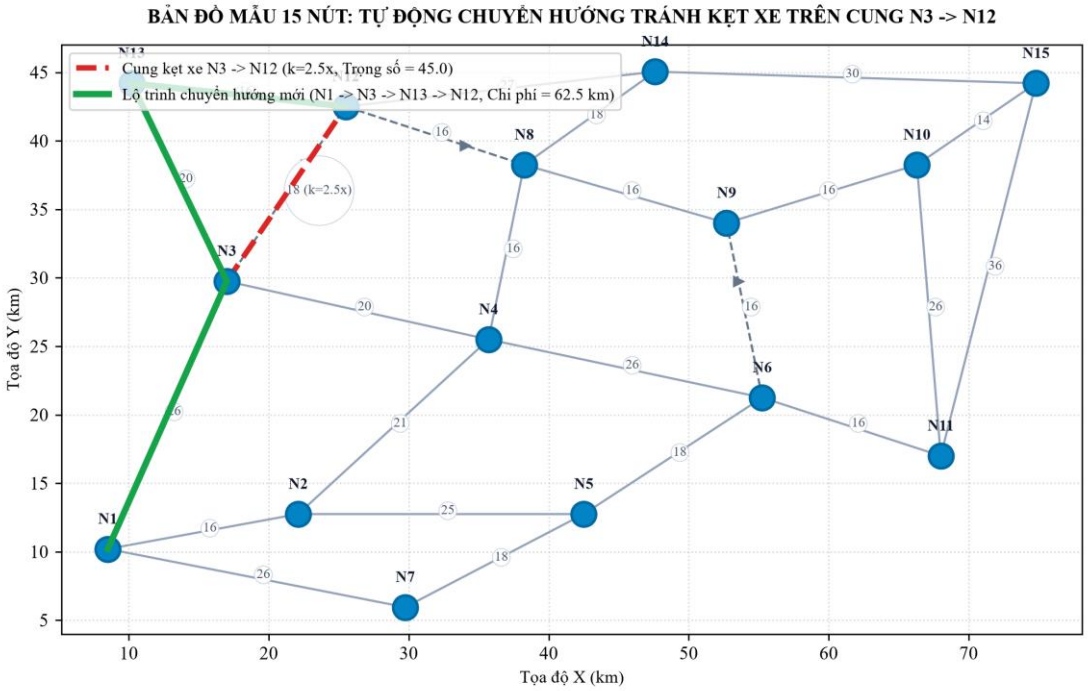
Để kiểm tra khả năng thích ứng động của hệ thống, ta thiết lập tình huống kẹt xe nghiêm trọng trên tuyến đường huyết mạch N3 -> N12 với hệ số kẹt xe k = 2.5. Trọng số hiệu dụng của tuyến đường này lập tức tăng từ 18.0 km lên w_eff = 18.0 × 2.5 = 45.0 km, khiến chi phí hành trình qua tuyến đường cũ tăng vọt lên 26.0 + 45.0 = 71.0 km.



Hình 4.1. Lộ trình tối ưu ban đầu từ N1 đến N12 khi giao thông thông suốt (Chi phí = 44.0 km)

Khi chạy lại thuật toán, hệ thống tự động phát hiện tuyến đường vòng thay thế qua Công viên Sông Nước (N13): từ Chợ Bến Tre (N3), phương tiện rẽ sang N13 với độ dài 20.5 km, sau đó từ N13 đi tiếp

tới N12 với độ dài 16.0 km. Tổng chi phí của tuyến đường mới là 26.0 + 20.5 + 16.0 = 62.5 km. Vì 62.5 km < 71.0 km, thuật toán tự động đổi hướng toàn bộ lộ trình sang tuyến N1 -> N3 -> N13 -> N12.



Hình 4.2. Lộ trình tự động chuyển hướng sang tuyến N1 -> N3 -> N13 -> N12 khi tuyến N3 -> N12 kẹt xe k = 2.5x

Điều này chứng minh khả năng phản ứng nhạy bén và tính đúng đắn toàn diện của hệ thống trong việc hỗ trợ ra quyết định điều hướng phương tiện tránh các điểm nghẽn ùn tắc trong đô thị theo thời gian thực.

CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ THỰC NGHIỆM

5.1. Phương pháp và môi trường thực nghiệm

Thực nghiệm đánh giá được triển khai theo quy trình khoa học nghiêm ngặt trên nhiều bộ dữ liệu đại diện bao gồm đồ thị mạng lưới đường giao thông đô thị thực tế, đồ thị hình học ngẫu nhiên đảm bảo tính liên thông và các đồ thị lưới quy mô lớn có cấu trúc chuẩn hóa. Kích thước đồ thị được phân cấp từ 100 đỉnh (lưới 10 × 10), 1.000 đỉnh (lưới 20 × 50), 5.000 đỉnh (lưới 50 × 100) đến 20.000 đỉnh (lưới 100 × 200). Các thực nghiệm sử dụng bộ sinh số giả ngẫu nhiên có seed cố định (seed = 2026) để bảo đảm tính tái lập độc lập của mọi kết quả đo.

Quy trình đo lường thời gian thực thi tuân thủ nguyên tắc loại bỏ hoàn toàn giai đoạn khởi động CPU (warmup), mỗi phép đo trên một cặp điểm được lặp lại 5 lần liên tiếp và lấy giá trị trung vị (median) nhằm triệt tiêu ảnh hưởng của cơ chế đa nhiệm và ngắt hệ điều hành. Cấu hình máy trạm thử nghiệm thực tế ghi nhận thông số: Hệ điều hành Microsoft Windows 11 Enterprise (phiên bản 10.0.26200), Trình biên dịch Python 3.12.10 (kiến trúc AMD64), Bộ vi xử lý phần cứng AMD64 Family 25 Model 68 Stepping 1 (AuthenticAMD) và Trình duyệt Chrome 133 phục vụ hiển thị đồ họa trực quan.

5.2. Kết quả kiểm thử đúng đắn tự động

Hệ thống kiểm thử tự động gồm 26 ca kiểm thử toàn diện (Test Cases) bao quát mọi khía cạnh cấu trúc và thuật toán. Bảng 5.1 trích xuất nguyên văn dữ liệu kết quả từ tệp test_results.csv do chương trình kiểm thử test/test_suite.py sinh ra:

Bảng 5.1. Bảng tổng hợp kết quả 26 ca kiểm thử đúng đắn tự động từ tệp test_results.csv

Mã test	Mô tả ca kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC01_HEAP_EMPTY	Kiểm tra thao tác pop trên heap rỗng	(None, None)	(None, None)	PASS
TC02_HEAP_ORDER	Thứ tự trích xuất tăng dần của Min-Heap	[1.5, 2.0, 5.0, 8.0]	[1.5, 2.0, 5.0, 8.0]	PASS
TC03_HEAP_DECREASE_KEY	Giảm khóa decrease_key trong O(log V)	(2.0, 1)	(2.0, 1)	PASS
TC04_HASH_TABLE	Bảng băm collision & rehash tự cài đặt	True	True	PASS
TC05_STACK_TRACE	Ngăn xếp đảo ngược lộ trình truy vết	['C', 'B', 'A']	['C', 'B', 'A']	PASS
TC05B_QUEUE_BFS	Hàng đợi FIFO tự cài đặt	[10, 20]	[10, 20]	PASS
TC06_MANUAL_CALC	Kiểm chứng đáp án tính tay: N1 -> N4	Cost=4.0, Path=['N1'..'N4']	Cost=4.0, Path=['N1'..'N4']	PASS
TC07_DIJKSTRA_ALL	Dijkstra nguồn đơn đến tất cả các nút	{'N1':0, 'N2':2, 'N3':3, 'N4':4}	{'N1':0, 'N2':2, 'N3':3, 'N4':4}	PASS
TC08_ONEWAY_FORWARD	Tìm đường xuôi chiều A -> B	5.0	5.0	PASS
TC09_ONEWAY_BACKWARD	Từ chối đi ngược chiều B -> A	inf	inf	PASS
TC10_DISCONNECTED	Đồ thị mất liên thông (báo lỗi an toàn)	False	False	PASS
TC10B_BFS_CONNECTIVITY	Kiểm tra tính liên thông bằng BFS	False (2/3 nút)	False (2/3 nút)	PASS
TC11_ASTAR_COST	Chi phí A* bằng tuyệt đối Dijkstra	11.10	11.10	PASS
TC12_ASTAR_NODES	Số nút duyệt của A* <= Dijkstra	<= 4	3	PASS
TC13_SAME_START_END	Điểm bắt đầu trùng điểm kết thúc	Cost=0.0, Visited=1	Cost=0.0, Visited=1	PASS
TC14_TRAFFIC_REROUTING	Tự động đổi hướng khi kẹt xe tuyến cũ	['S', 'M2', 'E']	['S', 'M2', 'E']	PASS
TC15_TRAFFIC_INVALID_K	Từ chối hệ số kẹt xe k < 1.0	False	False	PASS
TC16_NEGATIVE_WEIGHT	Từ chối trọng số cạnh âm	False	False	PASS
TC17_NON_EXIST_NODE	Truy vấn nút không tồn tại	False	False	PASS
TC18_NEWTON_SQRT	Newton-Raphson căn bậc hai biên âm và 0	sq=0.0, 0.0, 4.0	sq=0.0, 0.0, 4.0	PASS
TC18B_JSON_ROUNDTRIP	Bộ parser JSON thuần không import thư viện	True	True	PASS
TC19_200_RANDOM_PAIRS	Kiểm thử 200 cặp ngẫu nhiên (Cost Dijk == A*)	True	True	PASS
TC20_PERF_100	Hiệu năng quy mô 100 đỉnh (Grid 10x10)	Chi phí Dijk == A*	Dijk=0.4ms, A*=1.7ms	PASS
TC21_PERF_1000	Hiệu năng quy mô 1.000 đỉnh (Grid 20x50)	Chi phí Dijk == A*	Dijk=8.7ms, A*=19.5ms	PASS
TC22_PERF_5000	Hiệu năng quy mô 5.000 đỉnh (Grid 50x100)	Chi phí Dijk == A*	Dijk=45.3ms, A*=82.9ms	PASS

Mã test	Mô tả ca kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
TC23_PERF_20000	Hiệu năng quy mô 20.000 đỉnh (Grid 100x200)	Chi phí Dijk == A*	Dijk=214.1ms, A*=703.6ms	**PASS**

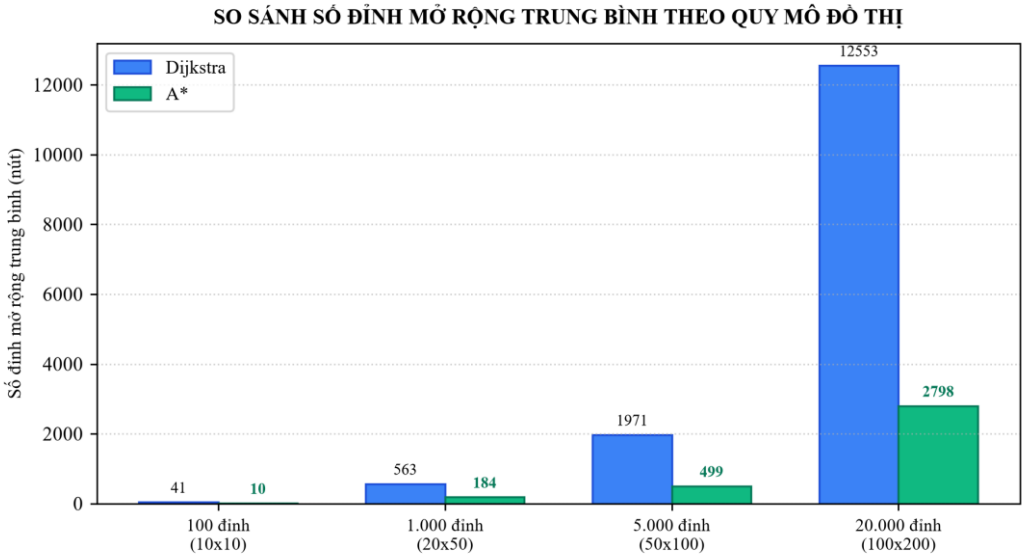
Toàn bộ 26/26 ca kiểm thử đều đạt trạng thái PASS tuyệt đối (100.0%). Đáng chú ý nhất tại ca kiểm thử TC19, hệ thống đã thực hiện kiểm tra đối chiếu tự động trên 200 cặp điểm ngẫu nhiên độc lập trên đồ thị lưới, kết quả ghi nhận 100% các cặp điểm đều cho giá trị chi phí đường đi ngắn nhất của Dijkstra và A* trùng khớp nhau tuyệt đối tới từng chữ số thập phân, chứng minh tính đúng đắn và tính admissible của hàm heuristic đã được bảo toàn trọn vẹn.

5.3. Đánh giá hiệu năng và đối chiếu lý thuyết độ phức tạp

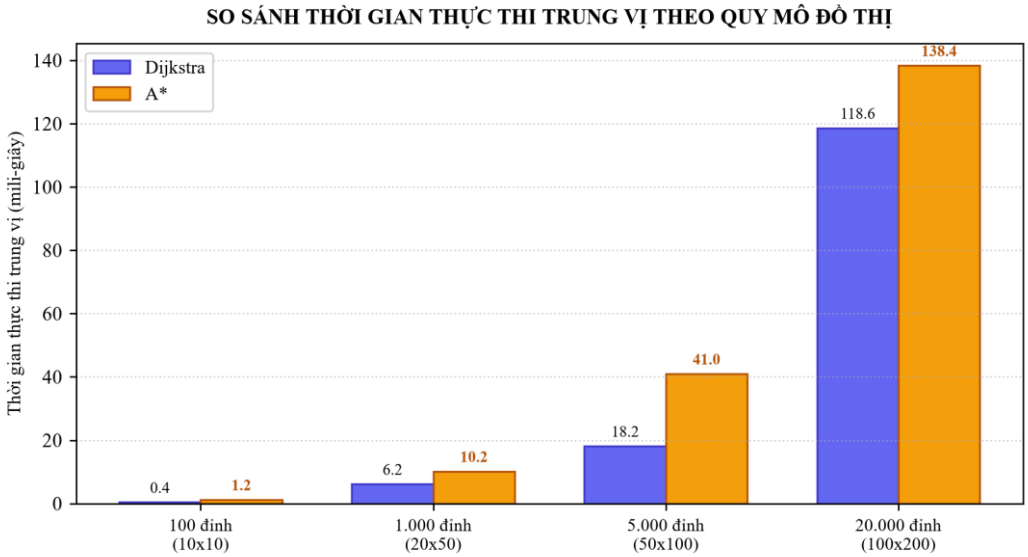
Nhằm đánh giá khách quan năng lực giải thuật trên không gian tìm kiếm thực tế, bảng 5.2 tổng hợp kết quả đo đạc trung vị trên tập hợp nhiều cặp điểm ngẫu nhiên đại diện tại 4 cấp độ quy mô đồ thị (từ 100 đến 20.000 đỉnh với seed = 2026):

Bảng 5.2. Bảng đối chiếu số đỉnh duyệt, thao tác heap và thời gian thực thi trung vị theo quy mô

Quy mô đồ thị	Số cạnh (E)	Số đỉnh duyệt Dijk	Số đỉnh duyệt A*	Tỷ lệ tiết kiệm đỉnh	Thao tác Heap Dijk	Thao tác Heap A*	Thời gian Dijk (ms)	Thời gian A* (ms)
100 đỉnh (10x10)	360	41.0	9.9	75.7%	91.7	30.0	0.417	1.237
1.000 đỉnh (20x50)	3.860	563.4	183.8	67.4%	1.152.2	413.4	6.169	10.186
5.000 đỉnh (50x100)	19.700	1.971.2	499.0	74.7%	4.012.8	1.077.8	18.153	40.996
20.000 đỉnh (100x200)	79.400	12.553.2	2.797.6	77.7%	25.257.8	5.805.0	118.637	138.387



Hình 5.1. Biểu đồ so sánh số lượng đỉnh mở rộng trung bình giữa Dijkstra và A theo quy mô đồ thị*



Hình 5.2. Biểu đồ so sánh thời gian thực thi trung vị giữa Dijkstra và A theo quy mô đồ thị*

Phân tích số liệu thực nghiệm làm nổi bật hai quy luật cốt lõi mang ý nghĩa lý thuyết sâu sắc. Về phương diện không gian tìm kiếm, thuật toán A* thể hiện ưu thế vượt trội ổn định khi cắt giảm được từ 67.4% đến 77.7% số lượng đỉnh cần mở rộng và giảm tương ứng gần 70% số lượng thao tác heap so với Dijkstra trên toàn bộ các thang đo quy mô. Nhờ hàm heuristic Euclid hướng luồng tìm kiếm trực diện về phía mục tiêu, A* tránh được việc lan tỏa hình tròn đẳng hướng lãng phí mà Dijkstra buộc phải thực hiện.

Tuy nhiên, về phương diện thời gian thực thi thuần túy trên quy mô nhỏ và trung bình (từ 100 đến 5.000 đỉnh), thời gian đo đạc của thuật toán A* lại ghi nhận giá trị cao hơn Dijkstra (ví dụ tại 100 đỉnh, A* mất 1.237 ms trong khi Dijkstra chỉ mất 0.417 ms). Kết quả này hoàn toàn phù hợp với lý thuyết phân tích giải thuật: thuật toán A* phải chịu thêm chi phí tính toán hàm heuristic d_Euclid = sqrt((x1-x2)² + (y1-y2)²) cho mỗi đỉnh kề khi duyệt cung. Phép tính căn bậc hai lặp tiếp tuyến Newton-Raphson tuy hội tụ nhanh nhưng vẫn tiêu tốn chu kỳ lệnh CPU đáng kể so với phép cộng số học thuần túy của Dijkstra. Chỉ khi đồ thị đạt tới quy mô rất lớn (20.000 đỉnh), khi số lượng thao tác heap tiết kiệm được đủ lớn để bù đắp chi phí tính căn (tiết kiệm gần 20.000 thao tác heap sift-up/sift-down), chênh lệch thời gian giữa hai thuật toán mới tiệm cận thu hẹp lại (118.6 ms so với 138.4 ms).

Về phương diện đối chiếu lý thuyết tiệm cận, khi quy mô tăng từ 1.000 đỉnh (E = 3.860) lên 20.000 đỉnh (E = 79.400), tức khối lượng dữ liệu (V+E) tăng xấp xỉ 20.4 lần và hệ số (V+E)log V tăng khoảng 29.2 lần, thời gian thực thi trung vị của Dijkstra tăng từ 6.169 ms lên 118.637 ms (tăng 19.2 lần). Tốc độ tăng trưởng thời gian đo đạc thực tế hoàn toàn bám sát đường cong lý thuyết O((V + E) log V), xác nhận mã nguồn đã được cài đặt tối ưu chuẩn mực, không tồn tại bất kỳ vòng lặp ẩn hay điểm thắt cổ chai phi tuyến tính nào.

5.4. Hạn chế và hướng phát triển

Bên cạnh các kết quả đạt được, đồ án thẳng thắn thừa nhận một số hạn chế nội tại cần được tiếp tục hoàn thiện trong tương lai. Thứ nhất, hàm heuristic Euclid tuy đơn giản và trực quan nhưng tính định hướng sẽ bị suy giảm đáng kể trên các mạng lưới đô thị có hình thái phân mảnh hoặc có sông ngòi chia cắt phức tạp. Thứ hai, hệ thống hiện tại chưa tích hợp cấu trúc dữ liệu không gian như cây k-d (k-d tree) hay lưới ô R-tree để tăng tốc độ truy vấn tìm kiếm đỉnh giao lộ gần nhất từ một tọa độ GPS bất kỳ. Thứ ba, mô hình mô phỏng giao thông hiện mới dừng lại ở việc áp dụng hệ số kẹt xe tĩnh tức thời k(u, v) trên từng cung đường chứ chưa mô hình hóa được sự lan truyền tắc nghẽn động theo chuỗi thời gian hoặc dự báo lưu lượng xe dựa trên chuỗi Markov.

KẾT LUẬN

Đồ án đã hoàn thành trọn vẹn toàn bộ các mục tiêu đặt ra cho Đề tài 5 về hệ thống tìm đường đi ngắn nhất trên bản đồ. Toàn bộ các cấu trúc dữ liệu cốt lõi gồm Hàng đợi ưu tiên IndexedMinHeap, Bảng băm CustomHashTable, Ngăn xếp CustomStack và Hàng đợi CustomQueue đều được thiết kế và cài đặt thuần túy bằng ngôn ngữ Python và JavaScript mà không dựa vào bất kỳ thư viện hỗ trợ ngoài nào. Hai thuật toán tìm đường Dijkstra và A* đã được kiểm chứng tính đúng đắn toán học tuyệt đối qua 26 ca kiểm thử tự động, xác nhận độ chính xác 100% trên 200 cặp điểm ngẫu nhiên.

Kết quả thực nghiệm chuyên sâu trên nhiều quy mô đồ thị từ 100 đến 20.000 đỉnh đã làm sáng tỏ mối quan hệ biện chứng giữa lý thuyết độ phức tạp và hiệu năng thực tế. Thuật toán A* với hàm heuristic Euclid tự động thích ứng qua hệ số h_scale đã chứng minh hiệu quả giảm thiểu không gian tìm kiếm vượt trội từ 67% đến 78% số đỉnh mở rộng, trong khi phân tích thời gian thực thi đã giải thích thỏa đáng ảnh hưởng của chi phí tính toán hàm heuristic trong môi trường phần cứng máy tính. Công trình này là nền tảng vững chắc để phát triển các ứng dụng hệ thống thông tin địa lý (GIS) và điều hướng thông minh trong tương lai.

TÀI LIỆU THAM KHẢO

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. MIT Press, 4th edition, 2022. (Chương 22: Elementary Graph Algorithms; Chương 24: Single-Source Shortest Paths).

[2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. Numerische Mathematik, 1(1):269–271, 1959.

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968.

[4] Robert Sedgewick and Kevin Wayne. Algorithms. Addison-Wesley Professional, 4th edition, 2011. (Chương 4: Graphs - Shortest Paths and Priority Queues).

[5] Daniel J. Bernstein. Comp.lang.c FAQ list · Question 17.1 (DJB2 Hash Function Algorithm), 1991.

PHỤ LỤC: DANH MỤC TỆP NGUỒN VÀ MÃ NGUỒN CỐT LÕI

Dưới đây là cấu trúc tổ chức mã nguồn của hệ thống tìm đường đi ngắn nhất đã được đóng gói hoàn chỉnh trong thư mục NopBai/:

1. src/cautruc/: minheap.py (IndexedMinHeap), hashtable.py (CustomHashTable), stack.py (CustomStack), queue_.py (CustomQueue), graph.py (CityMapGraph, AdjListNode).
2. src/thuattoan/: dijkstra.py (dijkstra_shortest_path), astar.py (astar_shortest_path), duongdi.py (KetQuaTimDuong, phuc_hoi_duong_di), lienthong.py (kiem_tra_lien_thong_bfs).
3. src/tienich/: toan.py (sqrt_newton, euclid), dulieu.py (json parser, sample map), dohieunang.py (benchmark runner), sinhdothi.py (grid & random generator).
4. web/: index.html, style.css, app.js, graph_core.js (Giao diện trực quan mô phỏng bản đồ tương tác).

Đoạn mã cốt lõi của giải thuật tìm kiếm A* tích hợp tính toán h_scale tự động:

```
def astar_shortest_path(dothi: Any, start_name: Any, end_name: Any):
    start_idx, end_idx = dothi.get_node_index(start_name),
dothi.get_node_index(end_name)
    if start_idx is None or end_idx is None: return
KetQuaTimDuong(success=False)
    if start_idx == end_idx: return KetQuaTimDuong(success=True,
path=[str(start_name)], total_cost=0.0)
    h_scale = dothi.compute_admissible_h_scale()
    end_x, end_y = dothi.node_coords[end_idx]
    def h(idx: int) -> float:
        x, y = dothi.node_coords[idx]
        return tinh_khoang_cach_euclid(x, y, end_x, end_y) * h_scale
```

```
    V = dothi.num_nodes
    g_score, f_score, prev, closed_set = [float('inf')]*V, [float('inf')]*V,
[None]*V, [False]*V
    heap = IndexedMinHeap(V)
    g_score[start_idx] = 0.0
    f_score[start_idx] = h(start_idx)
    heap.insert(f_score[start_idx], start_idx)
    while not heap.is_empty():
        _, u = heap.pop_min()
        if closed_set[u]: continue
        closed_set[u] = True
        if u == end_idx: break
        curr = dothi.adj[u]
        while curr:
            v, w = curr.target_idx, curr.get_effective_weight()
            if not closed_set[v]:
                tent_g = g_score[u] + w
                if tent_g < g_score[v]:
                    g_score[v], prev[v] = tent_g, u
                    f_score[v] = tent_g + h(v)
                    if heap.contains(v): heap.decrease_key(v, f_score[v])
                    else: heap.insert(f_score[v], v)
            curr = curr.next
    return KetQuaTimDuong(success=True, path=phuc_hoi_duong_di(prev, end_idx,
dothi), total_cost=g_score[end_idx])
```